

DataFrame di Pandas

Il **DataFrame di Pandas** è la struttura dati fondamentale in Python per l'analisi dei dati, assimilabile a una tabella bidimensionale con righe e colonne etichettate, molto simile a un foglio Excel o a una tabella SQL

1. Importazione e Creazione

```
import pandas as pd

# Creazione da un dizionario
dati = {
    'Nome': ['Alice', 'Bob', 'Charlie'],
    'Età': [25, 30, 35],
    'Città': ['Roma', 'Milano', 'Napoli']
}
df = pd.DataFrame(dati)

# risultato= df.to_csv('file.csv')
# risultato = df.to_excel('file.xlsx')
```

2. Esplorazione e Ispezione dei Dati

Prima di manipolare i dati, è fondamentale analizzarne la struttura e le prime righe per verificarne la correttezza.

- **df.head(n)**: Mostra le prime n righe del DataFrame (default 5).
 - **df.info()**: Mostra il tipo di dati delle colonne, i valori non nulli e l'uso della memoria.
 - **df.describe()**: Genera statistiche descrittive (media, deviazione standard, min, max) per le colonne numeriche.
 - **df.shape**: Restituisce una tupla con il numero di righe e di colonne (righe, colonne).
-

3. Selezione e Indicizzazione

La selezione permette di estrarre porzioni mirate di dati in base a colonne, righe o etichette.

- **Selezionare colonne:** `df['Nome']` (restituisce una Serie) o `df[['Nome', 'Città']]` (restituisce un DataFrame).
 - **Selezione basata su etichette (.loc):** `df.loc[0, 'Nome']` seleziona l'elemento alla riga con indice 0 e colonna 'Nome'.
 - **Selezione basata su posizioni intere (.iloc):** `df.iloc[0, 1]` seleziona l'elemento alla prima riga e seconda colonna.
-

4. Filtro dei Dati (Filtro Booleano)

È possibile estrarre righe specifiche applicando condizioni logiche vettorizzate.

Python

```
# Seleziona solo le righe dove l'Età è maggiore di 28
```

```
df_filtrato = df[df['Età'] > 28]
```

```
# Condizioni multiple (usare & per AND, | per OR)
```

```
df_multiplo = df[(df['Età'] > 20) & (df['Città'] == 'Roma')]
```

5. Manipolazione e Modifica

Le operazioni di pulizia e trasformazione permettono di aggiornare la struttura del set di dati.

- **Aggiungere una colonna:** `df['Stipendio'] = [2000, 2500, 3000]`.
 - **Eliminare righe o colonne (.drop):** `df.drop(columns=['Città'])` rimuove la colonna specificata.
 - **Gestione valori mancanti:** `df.dropna()` per eliminare le righe con valori nulli, oppure `df.fillna(valore)` per sostituirli.
 - **Rimuovere duplicati:** `df.drop_duplicates()` elimina le righe identiche.
-

6. Aggregazione e Raggruppamento

Il raggruppamento permette di segmentare i dati ed eseguire calcoli statistici su categorie specifiche

python

```
# Calcola la media dell'Età per ogni Città
```

```
media_per_citta = df.groupby('Città')['Età'].mean()
```

```
# Calcoli diretti su colonne
```

```
media_eta = df['Età'].mean()
```

```
somma_eta = df['Età'].sum()
```

1. Trovare i Nan

1. Contare i NaN per ogni colonna

Usa `isna()` (o `isnull()`) combinato con `sum()` per ottenere il numero esatto di valori mancanti in ciascuna colonna:

```
python  
print(df.isna().sum())
```

Usa il codice con cautela.

2. Filtrare e visualizzare le righe che contengono NaN

Se vuoi estrarre solo le righe in cui è presente almeno un valore NaN:

```
python  
righe_con_nan = df[df.isna().any(axis=1)]  
print(righe_con_nan)
```

Usa il codice con cautela.

3. Controllare se c'è almeno un NaN nell'intero DataFrame

Restituisce True se c'è anche solo un valore mancante:

```
python  
print(df.isna().any().any())
```

2. Sostituire i NaN

a. Sostituire tutti i NaN con un valore fisso

```
python  
  
# Sostituisce con lo zero  
df_pulito = df.fillna(0)  
  
# Sostituisce con una stringa (es. per colonne di testo)  
df_pulito = df.fillna("Mancante")
```

b. Sostituire con valori diversi per ogni colonna

Puoi passare un dizionario per specificare un valore di sostituzione diverso basato sul nome della colonna:

```
python  
valori_colonne = {'eta': 30, 'citta': 'Sconosciuta', 'punteggio': 0}  
df_pulito = df.fillna(value=valori_colonne)  
Usa il codice con cautela.
```

c. Sostituire con la Media, Mediana o Moda

Metodo comune nell'analisi dati per non alterare le statistiche descrittive delle colonne numeriche:

```
python  
# Sostituisce i NaN della colonna 'prezzo' con la sua media  
df['prezzo'] = df['prezzo'].fillna(df['prezzo'].mean())
```

d. Sostituire con il metodo Forward Fill o Backward Fill

Utile per le serie temporali, copia il valore adiacente precedente o successivo:

```
python  
# ffill: copia il valore della riga precedente  
df_pulito = df.fillna(method='ffill')  
  
# bfill: copia il valore della riga successiva  
df_pulito = df.fillna(method='bfill')
```

e. Nota importante sulle modifiche

Di base fillna() crea una copia del DataFrame. Per modificare direttamente il DataFrame originale senza riassegnarlo, usa il parametro inplace=True:

```
python  
df.fillna(0, inplace=True)
```

3. Per rimuovere i duplicati in un DataFrame di Pandas

import pandas as pd # Rimuove i duplicati mantenendo solo la prima occorrenza e sovrascrive il DataFrame df.drop_duplicates(inplace=True)

Parametri Principali

Il metodo accetta diversi parametri molto utili per personalizzare la rimozione:

- **subset:** permette di specificare una o più colonne da considerare per identificare il duplicato. Se omissso, controlla tutte le colonne.
- **keep:** stabilisce quale record mantenere.
 - 'first' (predefinito): mantiene la prima occorrenza ed elimina le successive.
 - 'last': mantiene l'ultima occorrenza ed elimina le precedenti.
 - False: elimina *tutte* le righe che hanno dei duplicati.
- **inplace:** se impostato su True, modifica il DataFrame originale senza crearne uno nuovo.
- **ignore_index:** se impostato su True, resetta l'indice numerico del DataFrame risultante da 0 a n-1.

Esempi Pratici

Rimuoviamo i duplicati basandoci esclusivamente sui valori univoci di una colonna specifica (es. 'ID_Utente'):

```
df.drop_duplicates(subset=['ID_Utente'], keep='first', inplace=True)
```

Se invece vuoi eliminare tutti i duplicati, inclusa la prima occorrenza (in modo che rimangano solo valori unici al 100%):

```
df.drop_duplicates(keep=False, inplace=True)
```

CODICE DI ESEMPIO PER GESTIONE DATASET E DATAFRAME

```
# esempio sui dataset e dataframe

#1. Importazione e Creazione
import pandas as pd
import os

def pulisci_console():
    # 'nt' indica Windows, 'posix' indica macOS e Linux
    os.system('cls' if os.name == 'nt' else 'clear')

# Creazione da un dizionario
dati = {
    'Nome': ['Alice', 'Bob',
    'Charlie', 'Nome1', 'Nome2', 'Nome3', 'Cognome', 'Cognome2', 'Cognome3'],
    'Età': [25, 30, 35, 20, 21, 22, 50, 51, 52],
    'Città': ['Roma', 'Milano',
    'Napoli', 'Catania', 'Catania', 'Catania', 'Salerno', 'Salerno', 'Salerno']
}
df = pd.DataFrame(dati)
# Caricamento da fonti esterne (es. CSV o Excel)
#risultato=df.to_csv ('./dataset_dataframe/file.csv')
#risultato = df.to_excel('./dataset_dataframe/file.xlsx')

#2. ISPEZINE DEL DATAFRAME
#peer default prende le prime 5
print(df.head())

#prendo le prime 7
print (df.head(7))

#tipo delle colonne
print(df.info())

#rstituisce alcune statistiche sui dati
#
print(df.describe())

print(df.shape)

#leggo dalla fine
print (df.tail(3))

pulisci_console()

#Selezione e Indicizzazione
print("Selezione e Indicizzazione")
Nomi=df['Nome'] # tutti i valori della colonna Nome e quindi è unaa SERIE
```

```

print(Nomi)

Nomi_Citta=df[['Nome', 'Città']]
print("questo è un datafram di 2 colonne")
print(Nomi_Citta)

#loc e i loc per selezionare la "cella"

#selezionare in base al nome della cella
valore1=df.loc[0, 'Nome']
valore2=df.loc[1, 'Nome']
print (valore1+"----"+valore2)

#selezionare in base ALLA POSIZIONE della cella
valore1=df.iloc[0, 0]
valore2=df.iloc[0, 1]

valore3=df.iloc[1, 0]
valore4=df.iloc[1, 1]

print (f"{valore1} ----{valore2}-----{valore3}-----{valore4}")# stringa formattata
aa prescindere dal tipo

#risultato = df.to_excel('./dataset_dataframe/dataframe.xlsx')
#risultato = df.to_csv('./dataset_dataframe/datafram.xlsx')

input("premi un tasto per continuare")
pulisci_console()

#Filtro dei Dati (Filtro Booleano)

# Seleziona solo le righe dove l'Età è maggiore di 28
df_risultato=df[df['Età'] > 28 ]
print (df_risultato)
#Seleziona solo le righe dove la città è salerno
df_risultato=df[df['Città'] == "Salerno"]
print (df_risultato)

pulisci_console()
#Manipolazione e Modifica
#• Aggiungere una colonna

df['Stipendio'] = [2000, 2500, 3000,1000,1000,1000,1000,1000,-1]

#df['Stipendio'] = [2000, 2500, 3000] QUESTA DA ERRORE PERCHE DEVE AVERE LA STESSA
DIMENSIONE DELLE ALTRE COLONNE
print(df)

#RIMUOVERE una colonna

```



```
df_modificato=df.drop(columns=['Stipendio'])
pulisci_console()
print("dataframe modificato ")
print(df_modificato)
print("-----")
print("---")
print("dataframe modioriginario")
print(df)
```

CODICE DI ESEMPIO PER LA GESTIONE DEEI VALORI NaN

```
Pulizia dei dati

import pandas as pd
import os

def pulisci_console():
    # 'nt' indica Windows, 'posix' indica macOS e Linux
    os.system('cls' if os.name == 'nt' else 'clear')

# Creazione da un dizionario
dati = {
    'Nome': ['Alice', 'Bob',
    'Charlie', 'Nome1', 'Nome2', 'Nome3', 'Cognome', 'Cognome2', 'Cognome3'],
    'Età': [25, 30, 35, 20, 21, 22, 50, 51, 52],
    'Città': ['Roma', 'Milano',
    'Napoli', 'Catania', 'Catania', 'Catania', 'Salerno', 'Salerno', 'Salerno']
}
df = pd.DataFrame(dati)

#1. rimuovere i duplicati in un DataFrame di Pandas
df.drop_duplicates(inplace=False) # conservo la prima occorrenza
print(df)

df_ripulito = df.drop_duplicates(subset=['Città'], keep='first',
inplace=False) # inplace=True modifica il dataframe di partenza
print(df_ripulito)
#df.drop_duplicates(keep=False, inplace=True) ELIMINA TUTTI I DUPLICATI

#Trovare i Nan
print(df.isna().sum()) # NUMERO ESATTO DI Nan

import numpy as np

# 1. Creiamo un DataFrame di esempio con alcuni valori NaN
dati = {
    "Nome": ["Alice", "Bob", "Charlie", "David", "Emma"],
    "Età": [25, np.nan, 30, np.nan, 22],
    "Stipendio": [50000, 60000, np.nan, 45000, np.nan],
    "Città": ["Roma", "Milano", "Napoli", None, "Torino"],
}

df = pd.DataFrame(dati)
print("--- DataFrame Originale ---")
print(df)
print("\nConteggio NaN per colonna:")
print(df.isna().sum())
```

```
#Controllare se c'è almeno un NaN nell'intero DataFrame
print(df.isna().any().any())

# 2. Esempio di ELIMINAZIONE (dropna)
# Eliminiamo le righe dove la Città è mancante
df_drop_citta = df.dropna(subset=["Città"])
print("\n--- DataFrame dopo dropna (rimossa riga 3 senza Città) ---")
print(df_drop_citta)

# 3. Esempio di SOSTITUZIONE (fillna) con la MEDIA
# Calcoliamo la media dell'età (escludendo i NaN in automatico)
media_eta = df["Età"].mean() # Risultato: 25.0

# Sostituiamo i NaN nella colonna Età
df["Età"] = df["Età"].fillna(media_eta)

# 4. Esempio di SOSTITUZIONE con un valore fisso o testuale
df["Stipendio"] = df["Stipendio"].fillna(0)
df["Città"] = df["Città"].fillna("Non specificata")

print("\n--- DataFrame Finale Pulito ---")
print(df)

#Sostituire con dizionario (valori diversi per colonne diverse)
valori_personalizzati = {'Age': 30, 'City': 'Sconosciuto'}
df_sostituito=df.fillna(value=valori_personalizzati)

print("\n--- DataFrame Finale Sostiutuzini ---")
print(df_sostituito)
input("premi un tasto")
```